

Tenth Practice ML

In this practice, we will learn:

- **Text Classification:**
 - How to use **Text Feature Extraction:**
 - **N-gram**
- **Decision Tree** and **Random Forest.**

Downloads, Imports, and Definitions

We update packages that their Colab version is too old.

```
# # upgrade plotly
# !pip install --upgrade plotly
```

```
Requirement already satisfied: plotly in
/usr/local/lib/python3.11/dist-packages (5.24.1)
Requirement already satisfied: tenacity>=6.2.0 in
/usr/local/lib/python3.11/dist-packages (from plotly) (9.0.0)
Requirement already satisfied: packaging in
/usr/local/lib/python3.11/dist-packages (from plotly) (24.2)
```

We import our regular packages.

```
# import numpy, matplotlib, etc.
import numpy as np
import pandas as pd
import seaborn as sns
import plotly.express as px
import matplotlib.pyplot as plt
import plotly.graph_objects as go

# sklearn imports
from sklearn import metrics
from sklearn import pipeline
from sklearn import linear_model
from sklearn import preprocessing
from sklearn import neural_network
from sklearn import model_selection
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import make_pipeline
from sklearn.feature_selection import RFECV
from sklearn.compose import ColumnTransformer
```

```
from sklearn.linear_model import SGDRegressor
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RepeatedKFold
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
```

Data Exploration

We use the updated [SMS Spam](#) dataset.



Overview

This dataset includes 5574 SMS messages.

Each sample (message) has 1 feature (it's text content) and a label ([spam](#) or [ham](#)).

Description

Each sample is one SMS message written in English.

The messages consists of letters, numbers and punctuations.

The target is a binary feature; **spam** for **spam** SMS messages and **ham** for **non-spam** SMS messages.



Let's download the dataset from Github and explore it with Pandas tools.

```
# download sms_spam.csv file from Github
!wget https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/sms_spam.csv

--2025-01-16 11:53:44--
https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/sms_spam.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
185.199.111.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|
185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 500336 (489K) [text/plain]
Saving to: 'sms_spam.csv.2'

 sms_spam.csv.2      0%[                  ]      0  ---KB/s
sms_spam.csv.2      100%[=====>] 488.61K  ---KB/s   in
0.04s

2025-01-16 11:53:44 (11.2 MB/s) - 'sms_spam.csv.2' saved
```

```
[500336/500336]
```

```
# load the sms_spam.csv file
sms_df = pd.read_csv('sms_spam.csv')
sms_df

{"summary": "{\n  \"name\": \"sms_df\",\n  \"rows\": 5574,\n  \"fields\": [\n    {\n      \"column\": \"type\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"spam\",\n          \"ham\"],\n        \"semantic_type\": \"\",\n        \"description\": \"\" }\n      },\n      {\n        \"column\": \"text\",\n        \"properties\": {\n          \"dtype\": \"string\",\n          \"num_unique_values\": 5160,\n          \"samples\": [\n            \"they released another Italian one today and it has a cosign option\",\n            \"I'm not. She lip synced with shangela.\"],\n          \"semantic_type\": \"\",\n          \"description\": \"\" }\n      }\n    ]\n  },\n  \"type\": \"dataframe\",\n  \"variable_name\": \"sms_df\"}
```

Text Feature Extraction

In text datasets, it is essential to extract features from the text, which requires encoding the text using various methods to represent its characteristics numerically.

One basic method for encoding text is the **word count** approach. This method involves counting the occurrences of each word in the text corpus. A corpus refers to a collection of text, often organized as sentences separated by commas or other delimiters. In our dataset, the text messages are already segmented. For this method, we take all the unique words in the dataset (e.g., "yes," "no," "hi," "hello") as features and count how many times each word appears in the text. These counts form the feature representation. This process is commonly referred to as vectorization and is conceptually similar to creating a histogram, where we count the frequency of each word.

Another approach is to extract features in a **semantic context**, considering the meaning of words in relation to their surrounding text. For instance, in the phrase "not friendly," the word "friendly" generally conveys a positive sentiment. However, when combined with "not," its meaning shifts to a negative sentiment. This method captures the contextual relationships between words to produce more meaningful feature representations.

We will use **Text Feature Extraction: N-gram**

N-gram

In the fields of computational linguistics and probability, an **n-gram** is a **contiguous sequence of n items from a given sample of text**.

The items can be phonemes, syllables, letters, words, or base pairs according to the application. The n-grams typically are collected from a text corpus.

Example Text:

Natural language processing is an emerging field of AI

1-gram words

- Natural
- language
- processing
- is
- an
- emerging
- field
- of
- AI

2-gram words

- Natural language
- language processing
- processing is
- is an
- an emerging
- emerging field
- field of
- of AI

```
# show sms_df info
```

```
sms_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5574 entries, 0 to 5573
```

```
Data columns (total 2 columns):
```

#	Column	Non-Null Count	Dtype
0	type	5574 non-null	object
1	text	5574 non-null	object

```
dtypes: object(2)
```

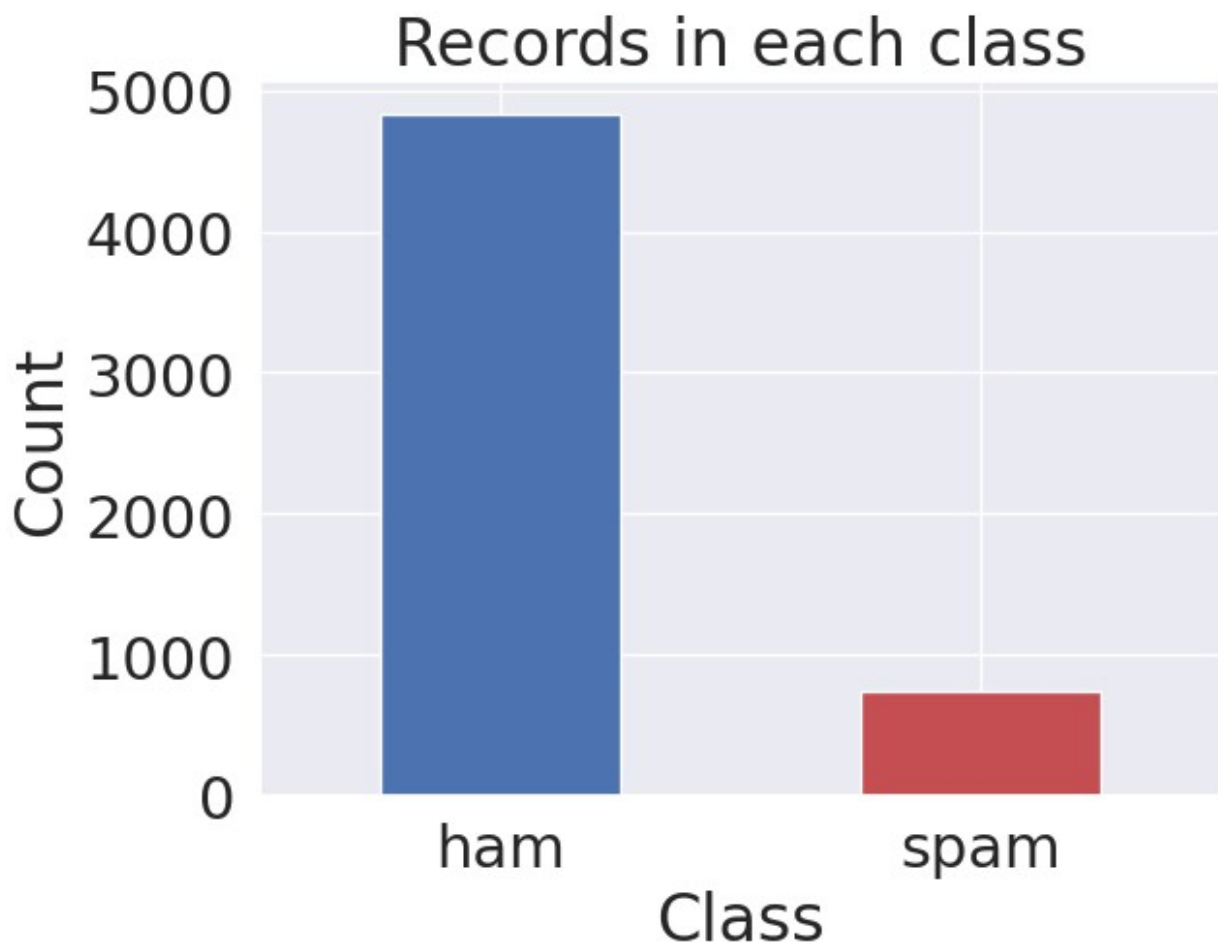
```
memory usage: 87.2+ KB
```

```
# show sms_df description
```

```
sms_df.describe()
```

```
{"summary":{"name":"sms_df","rows":4,"fields":4,"column":"type","properties":{"dtype":"string","num_unique_values":4,"samples":[2,4827,5574],"semantic_type":"","description":"","column":"","text":"","properties":{"dtype":"string",
```

```
count_classes = sms_df['type'].value_counts()
plt.title('Records in each class')
count_classes.plot(kind='bar', color=['b', 'r'])
plt.xticks(rotation='horizontal')
plt.xlabel('Class')
plt.ylabel("Count")
plt.show()
```



```
INFO: SPAM or HAM
type
ham      4827
```

```

spam      747
Name: count, dtype: int64
type
ham      0.865985
spam     0.134015
Name: proportion, dtype: float64

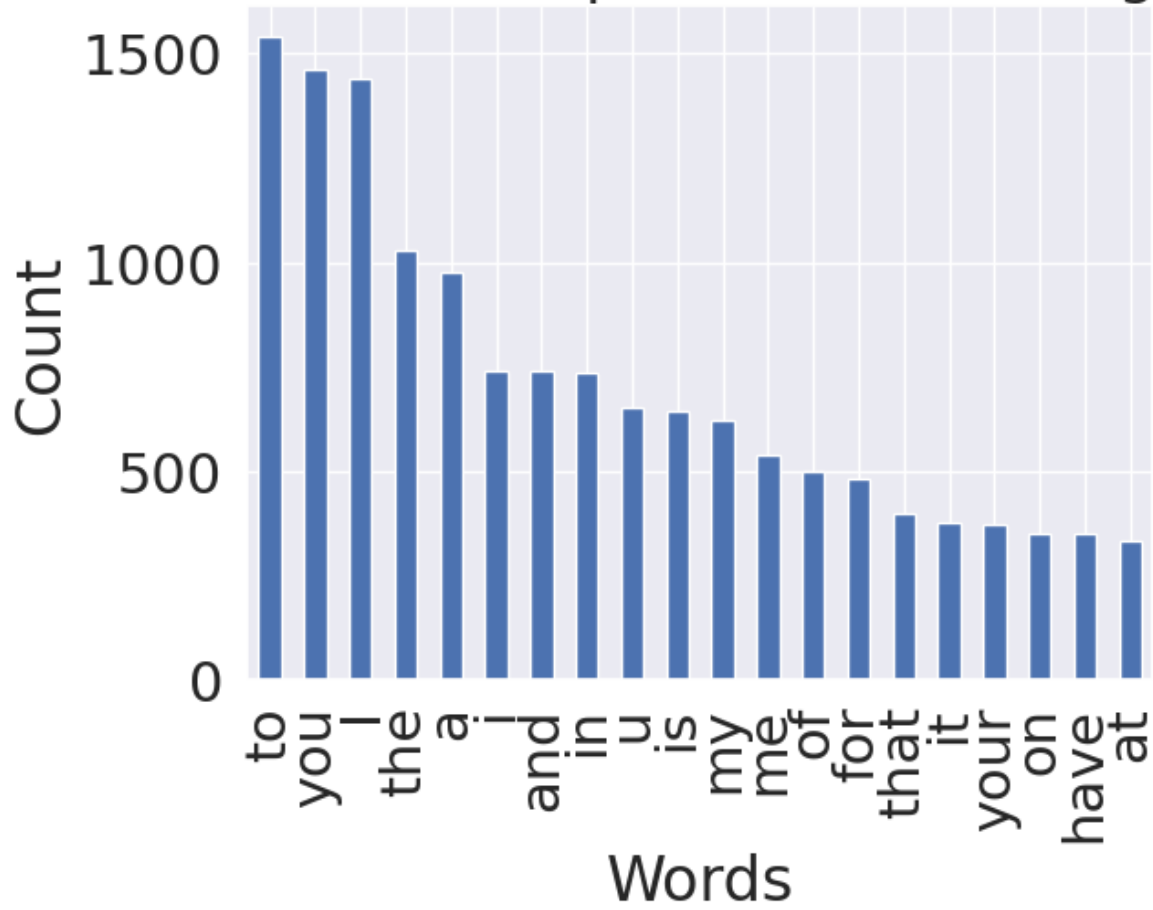
from collections import Counter
def plot_most_common(count, class_name, color):
    count_res = Counter(" ".join(sms_df[sms_df["type"] == class_name]
["text"]).split()).most_common(count)
    # convert to dataframe
    df_count = pd.DataFrame.from_dict(count_res)
    # rename columns
    df_count = df_count.rename(columns={0: class_name, 1: 'count'})

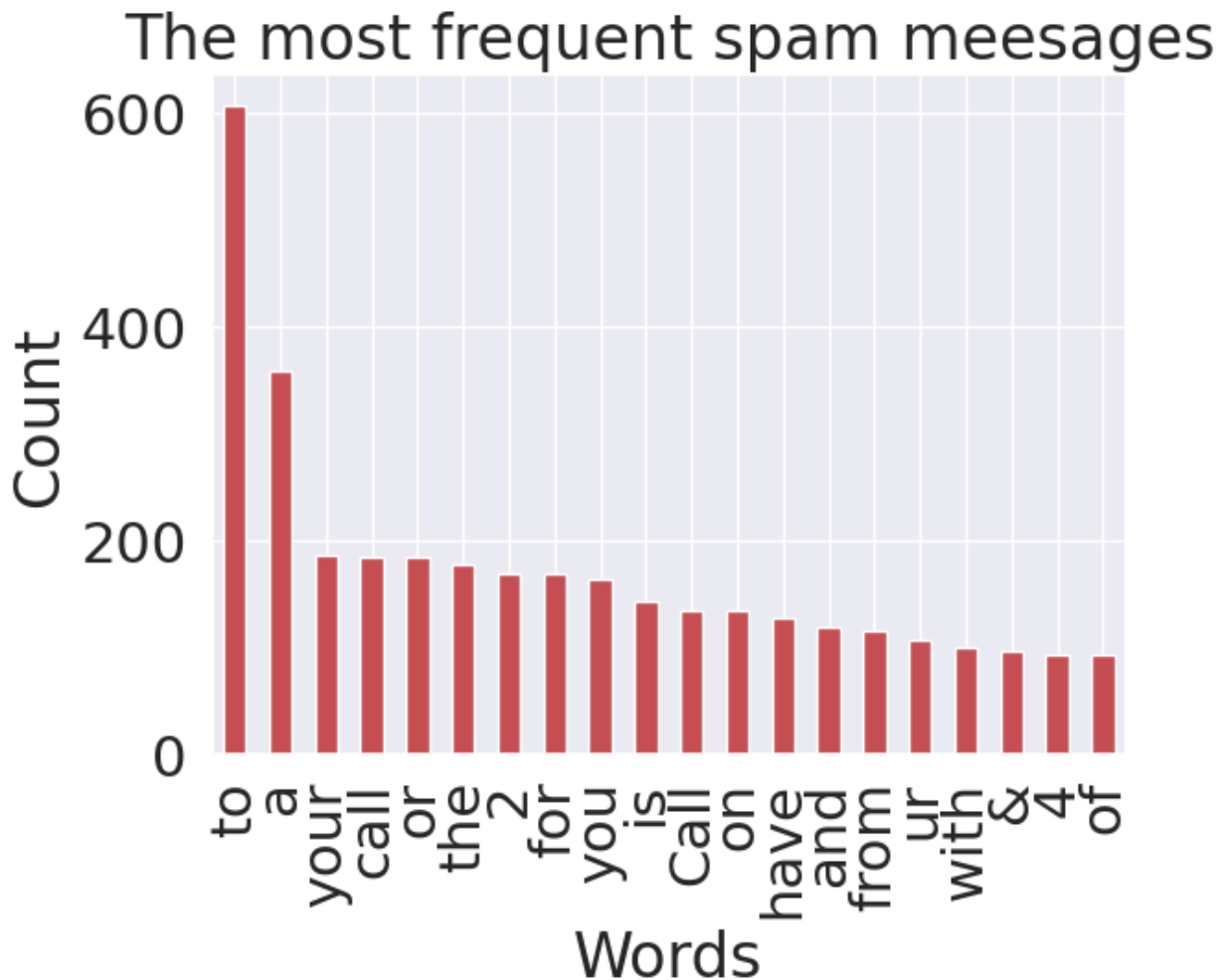
    # draw bar plot
    df_count.plot(kind='bar', color=color, legend=False)
    x = np.arange(len(df_count[class_name]))
    plt.xticks(x, df_count[class_name])
    plt.title(f'The most frequent {class_name} meesages')
    plt.xlabel('Words')
    plt.ylabel('Count')

# call the function twice (for 'ham' and for 'spam' classes
respectively)
plot_most_common(20, 'ham', 'b')
plot_most_common(20, 'spam', 'r')
plt.show()

```

The most frequent ham meessages





CountVectorizer Documentation:

Convert a collection of text documents to a matrix of token counts.

`ngram_range`: tuple (min_n, max_n), default=(1, 1)

The lower and upper boundary of the range of n-values for different word n-grams or char n-grams to be extracted. All values of n such that $\text{min_n} \leq n \leq \text{max_n}$ will be used. For example an `ngram_range` of (1, 1) means only unigrams, (1, 2) means unigrams and bigrams, and (2, 2) means only bigrams. Only applies if analyzer is not callable.

`analyzer`{'word', 'char', 'char_wb'} or callable, default='word'

Whether the feature should be made of word n-gram or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space.

If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input.

Let's start using the N-gram with Scikit-learn [CountVectorizer](#).
We can use 1-gram of words.

```
# extract 1-gram words from the data
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer() # counts the number of times each word
appears
messages = ['Hi there, I love Afeka college 2025', 'Hi there, I love
HIT college 2025']
v = vectorizer.fit_transform(messages) # separates the messages with a
coma
_df = pd.DataFrame(v.toarray(),
columns=vectorizer.get_feature_names_out())
_df

{"summary": "{\n  \"name\": \"_df\", \n  \"rows\": 2, \n  \"fields\": [\n    {\n      \"column\": \"2025\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 1, \n        \"num_unique_values\": 1, \n        \"samples\": [\n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      \"column\": \"afeka\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 0, \n        \"max\": 1, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          0\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      \"column\": \"college\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 1, \n        \"num_unique_values\": 1, \n        \"samples\": [\n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      \"column\": \"hi\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 1, \n        \"num_unique_values\": 1, \n        \"samples\": [\n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      \"column\": \"love\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 1, \n        \"num_unique_values\": 1, \n        \"samples\": [\n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      \"column\": \"there\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 1, \n        \"num_unique_values\": 1, \n        \"samples\": [\n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    ]\n  }, \n  \"type\": \"dataframe\"}
```

Note: The word "I" is not considered in the analysis. The Count Vectorizer only includes words with more than two characters.

```
# stop word - words that do not contribute meaningful information to
the context of the sentence are excluded.
vectorizer = CountVectorizer(stop_words='english') # 'a', 'there',
etc. are stop words (words without meaning called stop words)
messages = ['Hi there, I love Afeka college 2025', 'Hi there, I love
HIT college 2025']
v = vectorizer.fit_transform(messages) # separetes the messages with a
coma
_df = pd.DataFrame(v.toarray(),
columns=vectorizer.get_feature_names_out())
_df

{"summary": "{\n  \"name\": \"_df\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n      \"column\": \"2025\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n        \"max\": 1,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"afeka\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"college\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n        \"max\": 1,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"hi\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n        \"max\": 1,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"love\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n        \"max\": 1,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}
```

Remove stop words

```
# convert a collection of text documents to a feature matrix
vectorizer = CountVectorizer(stop_words='english')
X = vectorizer.fit_transform(sms_df['text'])

# convert to array
X = X.toarray()
X.shape

(5574, 8444)

X[:5]

array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]])

sms_df['type'] = sms_df['type'].map({'ham': 'not_spam', 'spam':
'spam'})
sms_df

{"summary": "{\n  \"name\": \"sms_df\",\n  \"rows\": 5574,\n  \"fields\": {\n    \"column\": \"type\",\n    \"properties\": {\n      \"dtype\": \"category\",\n      \"num_unique_values\": 2,\n      \"samples\": [\n        \"spam\",\n        \"not_spam\"\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    }\n  },\n  \"column\": \"text\",\n  \"properties\": {\n    \"dtype\": \"string\",\n    \"num_unique_values\": 5160,\n    \"samples\": [\n      \"they released another Italian one today\nand it has a cosign option\",\n      \"I'm not. She lip synced\nwith shangela.\"\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\"\n  }\n}", "type": "dataframe", "variable_name": "sms_df"}

sms_df['type'].value_counts()

type
not_spam    4827
spam        747
Name: count, dtype: int64
```

Let's divide the data frame into features and targets.

```
# split dataset to train and test data
X_train, X_test, t_train, t_test = train_test_split(X, sms_df['type'],
test_size=0.2, random_state=21)
```

```

# min max normalization
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))
X_train = scaler.fit_transform(X_train.copy())
X_test = scaler.fit_transform(X_test.copy())

print(f' X train:\n {X_train}')
print()
print(f'X test:\n {X_test}')
print()
print(f'y train:\n {t_train}')
print()
print(f't test:\n {t_test}')

X train:
[[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]

X test:
[[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]

y train:
1199    not_spam
4339    not_spam
4021    not_spam
1860    not_spam
2651    not_spam
...
4706    not_spam
1144    not_spam
48      not_spam
772     not_spam
5327    not_spam
Name: type, Length: 4459, dtype: object

t test:
5178    not_spam
5399    not_spam

```

```

3570    not_spam
4919    not_spam
5079     spam
...
990     not_spam
1585    not_spam
2877    not_spam
2562    not_spam
3922    not_spam
Name: type, Length: 1115, dtype: object

```

```

from sklearn.metrics import recall_score, precision_score,
confusion_matrix

```

```

def display_train_val_cms(title, t_train, t_test, y_train, y_test):
    cm_train = confusion_matrix(t_train, y_train)
    cm_test = confusion_matrix(t_test, y_test)
    cm_train_norm = confusion_matrix(t_train, y_train, normalize='true')
    cm_test_norm = confusion_matrix(t_test, y_test, normalize='true')
    cm_train_df_norm = pd.DataFrame(cm_train_norm,
index=['actual_not_spam', 'actual_spam'],
columns=['predicted_not_spam', 'predicted_spam'])
    cm_test_df_norm = pd.DataFrame(cm_test_norm,
index=['actual_not_spam', 'actual_spam'],
columns=['predicted_not_spam', 'predicted_spam'])
    cm_train_df = pd.DataFrame(cm_train, index=['actual_not_spam',
'actual_spam'], columns=['predicted_not_spam', 'predicted_spam'])
    cm_test_df = pd.DataFrame(cm_test, index=['actual_not_spam',
'actual_spam'], columns=['predicted_not_spam', 'predicted_spam'])
    recall_precision_train = f'Recall = {recall_score(t_train, y_train,
pos_label="spam"):3f}, Precision = {precision_score(t_train, y_train,
pos_label="spam"):3f}'
    recall_precision_test = f'Recall = {recall_score(t_test, y_test,
pos_label="spam"):3f}, Precision = {precision_score(t_test, y_test,
pos_label="spam"):3f}'
    sns.set(font_scale=2)
    fig = plt.figure(figsize=(30, 12))
    plt.suptitle(title, fontsize=32)

    ax1 = fig.add_subplot(221)
    sns.heatmap(cm_train_df, annot=True, cmap=plt.cm.Pastel1, fmt='g')
    ax1.title.set_text(f"Train: {recall_precision_train}")

    ax2 = fig.add_subplot(222)
    sns.heatmap(cm_test_df, annot=True, cmap=plt.cm.Pastel2, fmt='g')
    ax2.title.set_text(f"Test: {recall_precision_test}")

    ax3 = fig.add_subplot(223)
    sns.heatmap(cm_train_df_norm, annot=True, cmap=plt.cm.Pastel1,
fmt='g')

```

```
# ax3.title.set_text(f"Train: {recall_precision_train}")

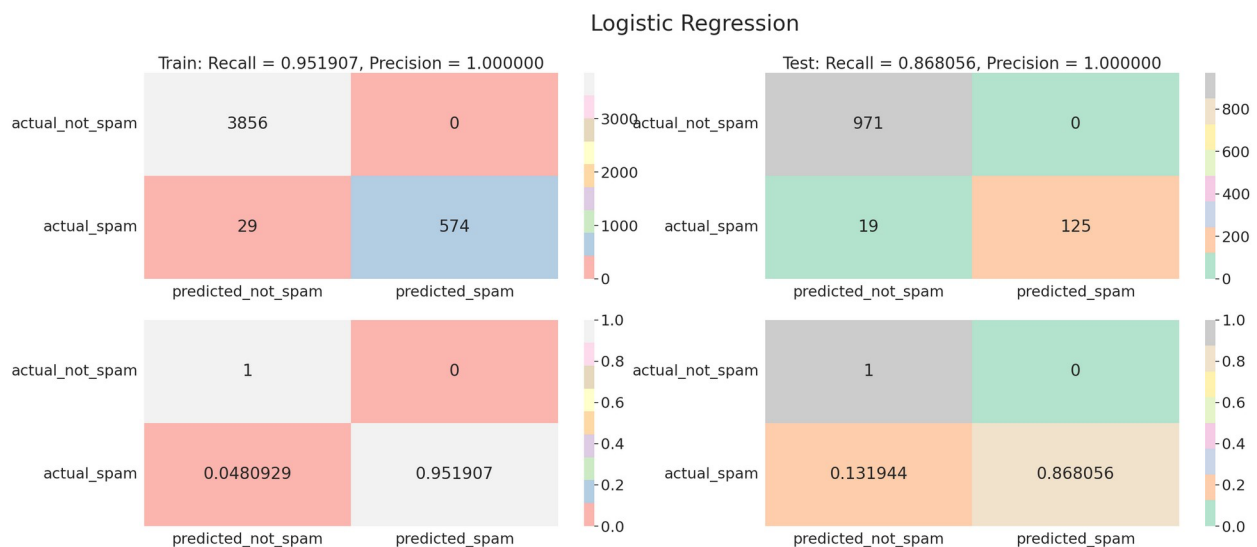
ax4 = fig.add_subplot(224)
sns.heatmap(cm_test_df_norm, annot=True, cmap=plt.cm.Pastel2,
fmt='g')
# ax4.title.set_text(f"Train: {recall_precision_train}")
plt.show()
```

Model 1: Logistic Regression Classifier

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression()
clf.fit(X_train, t_train)
y_train = clf.predict(X_train)
y_test = clf.predict(X_test)
print(f'Test counter = {Counter(y_test)}')
_c = Counter(t_test)
total = sum(_c.values(), 0.0)
for key in _c:
    _c[key] /= total

print(f'Spam / Not spam ratio on test: Spam = {_c["spam"]}, Not Spam =
{_c["not_spam"]}')
display_train_val_cms('Logistic Regression', t_train, t_test, y_train,
y_test)
```

Test counter = Counter({'not_spam': 990, 'spam': 125})
Spam / Not spam ratio on test: Spam = 0.12914798206278028, Not Spam = 0.8708520179372198



Model 2: NBC

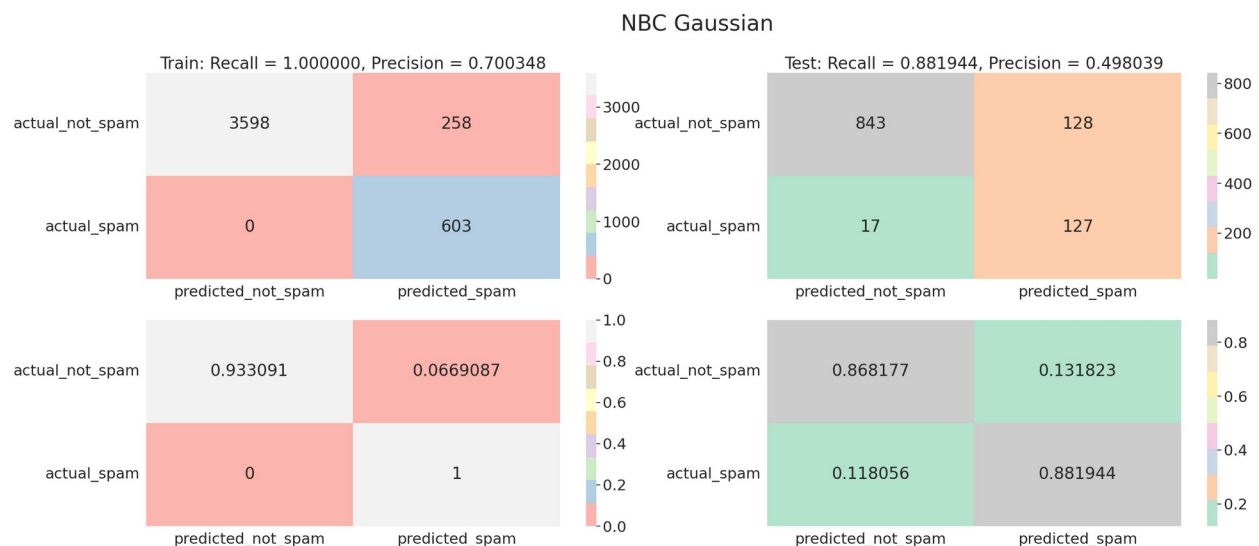
```
# create Gaussian model (demonstrateion for unsuitable model)
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, t_train)
y_train = model.predict(X_train)
y_test = model.predict(X_test)

print(f'Test counter = {Counter(y_test)}')
_c = Counter(t_test)
total = sum(_c.values(), 0.0)
for key in _c:
    _c[key] /= total

print(f'Spam / Not spam ratio on test: Spam = {_c["spam"]}, Not Spam =
{_c["not_spam"]}')
display_train_val_cms('NBC Gaussian', t_train, t_test, y_train,
y_test)

Test counter = Counter({'not_spam': 860, 'spam': 255})
Spam / Not spam ratio on test: Spam = 0.12914798206278028, Not Spam =
0.8708520179372198
```



```
# create NB multinomial model
from sklearn.naive_bayes import MultinomialNB
from collections import Counter

model = MultinomialNB()
model.fit(X_train, t_train)
y_train = model.predict(X_train)
y_test = model.predict(X_test)
```



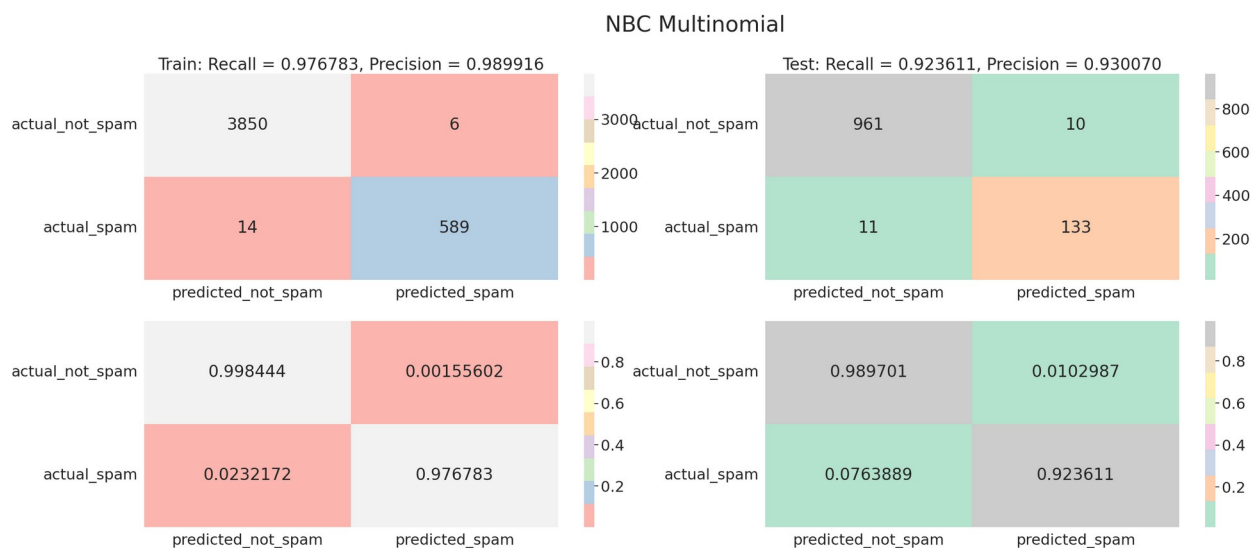
```

print(f'Test counter = {Counter(y_test)}')
_c = Counter(t_test)
total = sum(_c.values(), 0.0)
for key in _c:
    _c[key] /= total

print(f'Spam / Not spam ratio on test: Spam = {_c["spam"]}, Not Spam =
{_c["not_spam"]}')
display_train_val_cms('NBC Multinomial', t_train, t_test, y_train,
y_test)

Test counter = Counter({'not_spam': 972, 'spam': 143})
Spam / Not spam ratio on test: Spam = 0.12914798206278028, Not Spam =
0.8708520179372198

```



Why we didn't check the accuracy?

Decision Tree

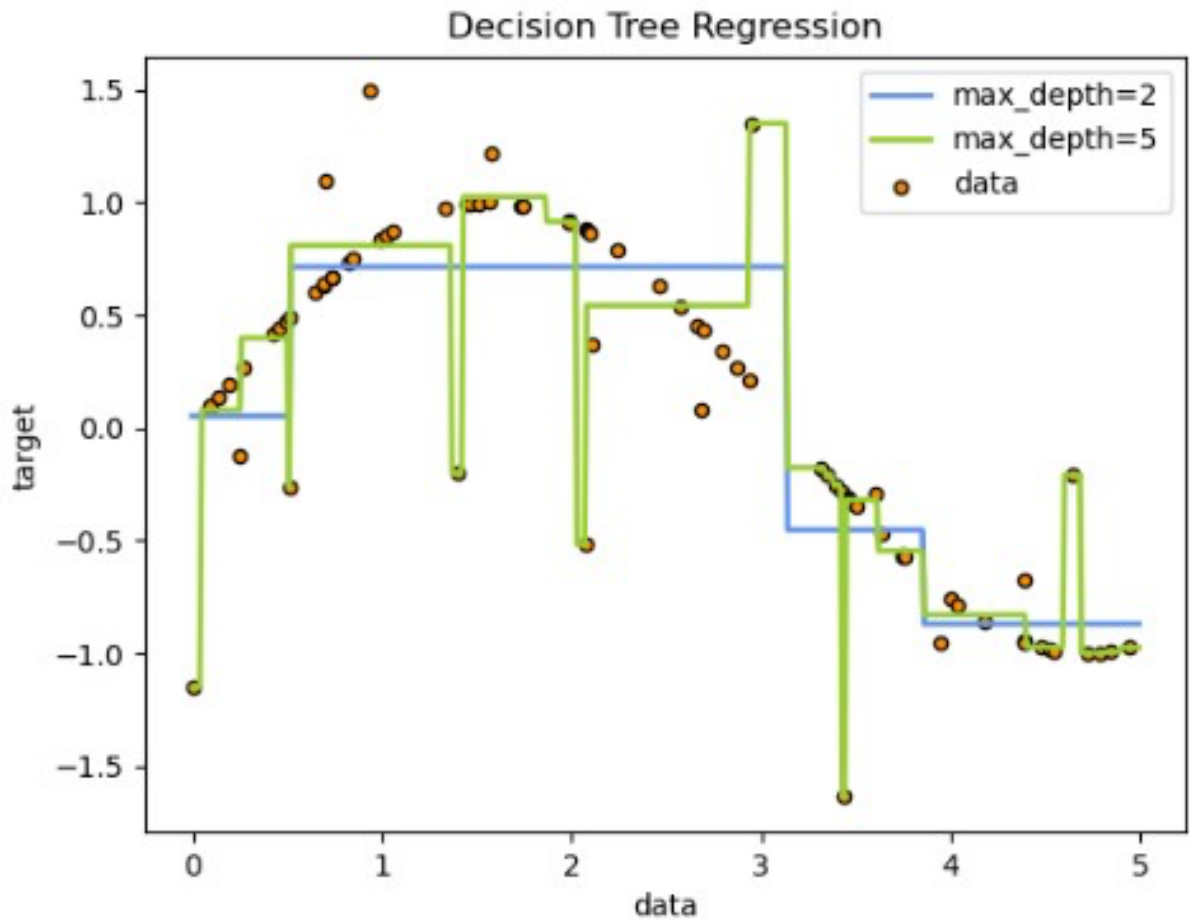
Decision Trees (DTs) are a non-parametric supervised learning method used for **classification** and **regression**.

The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

For instance, in the example below, decision trees learn from data to approximate a sine curve with a set of if-then-else decision rules.

The deeper the tree, the more complex the decision rules and the fitter the model (may cause

over-fitting):



Metrics

When using decision trees, we use heuristics to decide which node will be next. Our objective function is to **maximize** the metric at each split (the IG - Information Gain).

IG (Information Gain)

IG is defined as:

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^m \frac{N_j}{N} I(D_j),$$

where f is the feature to perform the split, and D_p and D_j are the datasets of the parent and j th child node, respectively.

I is the impurity measure (Entropy or Gini).

N is the total number of samples, and N_j is the number of samples at the j th child node.

We may use two metrics as I :

1. Entropy
2. Gini

Entropy

Entropy is defined as:

$$I_H(t) = - \sum_{i=1}^c p(i|t) \log_2 p(i|t),$$

$p(i|t)$ is the proportion of the samples that belong to class i for a particular node t .

The entropy is therefore 0 if all samples at a node belong to the same class, and the Entropy is maximal if we have a uniform class distribution.

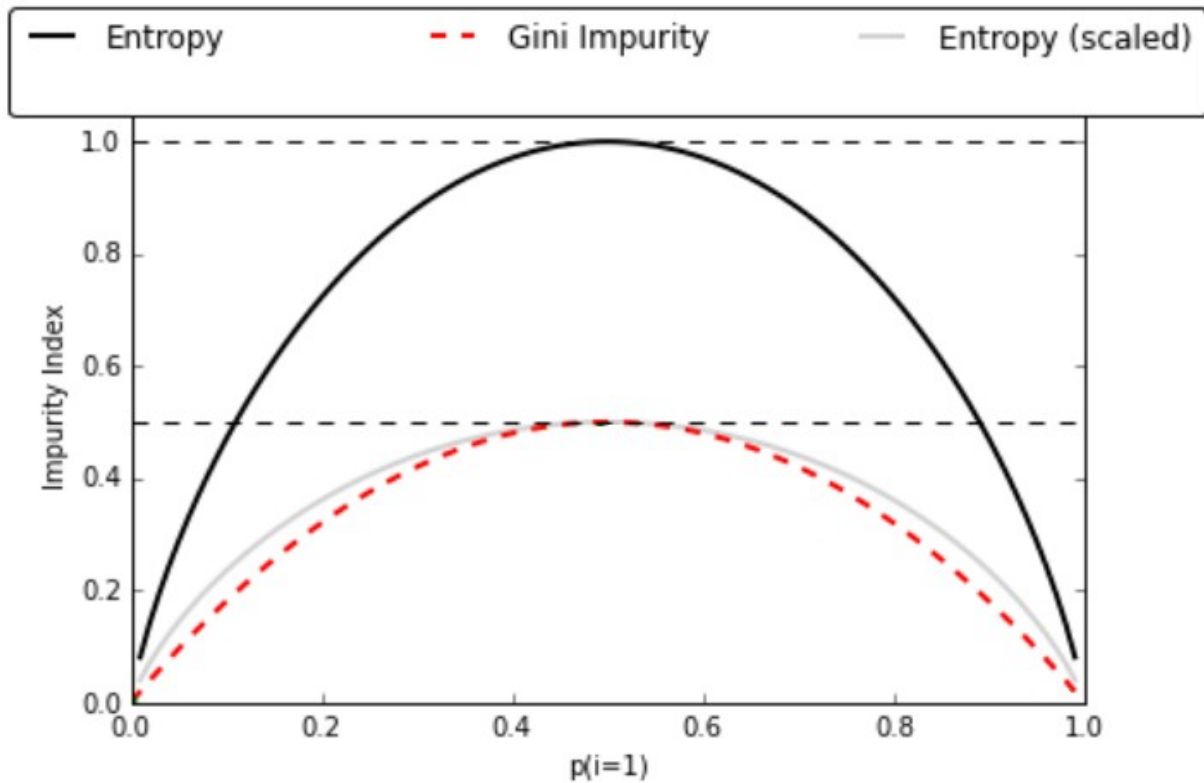
Gini

Gini is defined as:

$$I_G(t) = \sum_{i=1}^c p(i|t)(1 - p(i|t)) = 1 - \sum_{i=1}^c p(i|t)^2.$$

Similar to the Entropy, the Gini Impurity is maximal if the classes are perfectly mixed.

Comparison



Note: In practice, both Gini Impurity and Entropy typically yield very similar results and it is often not worth spending much time on evaluating trees using different impurity criteria rather than experimenting with different pruning cut-offs.

Some advantages of decision trees are:

- Simple to understand and to interpret. Trees can be visualized.
- Requires little data preparation.
- The cost of using the tree (i.e., predicting data) is logarithmic in the number of data points used to train the tree (the tree height).
- Able to handle both numerical and categorical data.
- Able to handle multi-output problems.
- Uses a white-box model. If a given situation is observable in a model, the explanation for the condition is easily explained by boolean logic. By contrast, in a black-box model (e.g., in an artificial neural network), results may be more difficult to interpret.

The disadvantages of decision trees include:

- Decision-tree learners can create over-complex trees that do not generalize the data well. This is called overfitting. Mechanisms such as pruning, setting the minimum number of samples required at a leaf node, or setting the maximum depth of the tree are necessary to avoid this problem.
- Decision trees can be unstable because small variations in the data might result in a completely different tree being generated. This problem is mitigated by using decision trees within an ensemble.
- Predictions of decision trees are neither smooth nor continuous, but piecewise constant approximations as seen in the above figure. Therefore, they are not good at extrapolation (infer something new from the data).
- The problem of learning an optimal decision tree is known to be NP-complete under several aspects of optimality and even for simple concepts. Consequently, practical decision-tree learning algorithms are based on heuristic algorithms such as the greedy algorithm where locally optimal decisions are made at each node. Such algorithms cannot guarantee to return the globally optimal decision tree. This can be mitigated by training multiple trees in an ensemble learner, where the features and samples are randomly sampled with replacement.
- Decision tree learners create biased trees if some classes dominate. It is therefore recommended to balance the dataset before fitting with the decision tree.

Let's use Scikit-learn [DecisionTreeClassifier](#).

```
from sklearn import datasets
data_dict = datasets.load_iris()
iris_df = pd.DataFrame(data=data_dict['data'],
columns=data_dict['feature_names'])
iris_df['class'] = data_dict['target']
iris_df

{"summary":{"\n  \"name\": \"iris_df\",\n  \"rows\": 150,\n  \"fields\": [\n    {\n      \"column\": \"sepal length (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n        \"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\": 35,\n        \"samples\": [\n          6.2,\n          4.5,\n          5.6\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"sepal width (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.435866284936698,\n        \"min\": 2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n        \"samples\": [\n          2.3,\n          4.0,\n          3.5\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"petal length (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.7652982332594667,\n        \"min\": 1.0,\n        \"max\": 6.9,\n
```

```

{"num_unique_values": 43,\n      "samples": [\n      6.7,\n      3.8,\n      3.7\n      ],\n      "semantic_type": "\"",\n      "description": "\""\n      },\n      {\n      "column":\n      "petal width (cm)",\n      "properties": {\n      "dtype":\n      "number",\n      "std": 0.7622376689603465,\n      "min":\n      0.1,\n      "max": 2.5,\n      "num_unique_values": 22,\n      "samples": [\n      0.2,\n      1.2,\n      1.3\n      ],\n      "semantic_type": "\"",\n      "description": "\""\n      },\n      {\n      "column": "class",\n      "properties": {\n      "dtype": "number",\n      "std": 0,\n      "min": 0,\n      "max": 2,\n      "num_unique_values": 3,\n      "samples": [\n      0,\n      1,\n      2\n      ],\n      "semantic_type": "\"",\n      "description": "\""\n      }\n      }\n      ],\n      "type": "dataframe",\n      "variable_name": "iris_df"}

```

word show accuracy of tree classifier on data

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score

```

take the data between the first and the third quarter

```

q1 = iris_df['sepal width (cm)'].quantile(0.25)
q3 = iris_df['sepal width (cm)'].quantile(0.75)
iqr = q3 - q1
iris_df = iris_df[(iris_df['sepal width (cm)'] >= q1 - 0.5*iqr) &
(iris_df['sepal width (cm)'] <= q3 + 1.5*iqr)]

t = iris_df['class']
X = iris_df.drop('class', axis=1)
X_train, X_test, t_train, t_test = model_selection.train_test_split(X,
t, test_size=0.25, random_state=2)

```

```

dt = DecisionTreeClassifier(max_depth=3, min_samples_leaf=10,
random_state=42)

```

dt = DecisionTreeClassifier(max_depth=3, random_state=42)

```
dt.fit(X_train, t_train)
```

```
DecisionTreeClassifier(max_depth=3, random_state=42)
```

```

from IPython.display import Image
from sklearn.tree import export_graphviz

```

```

!pip install pydotplus
import pydotplus

```

```

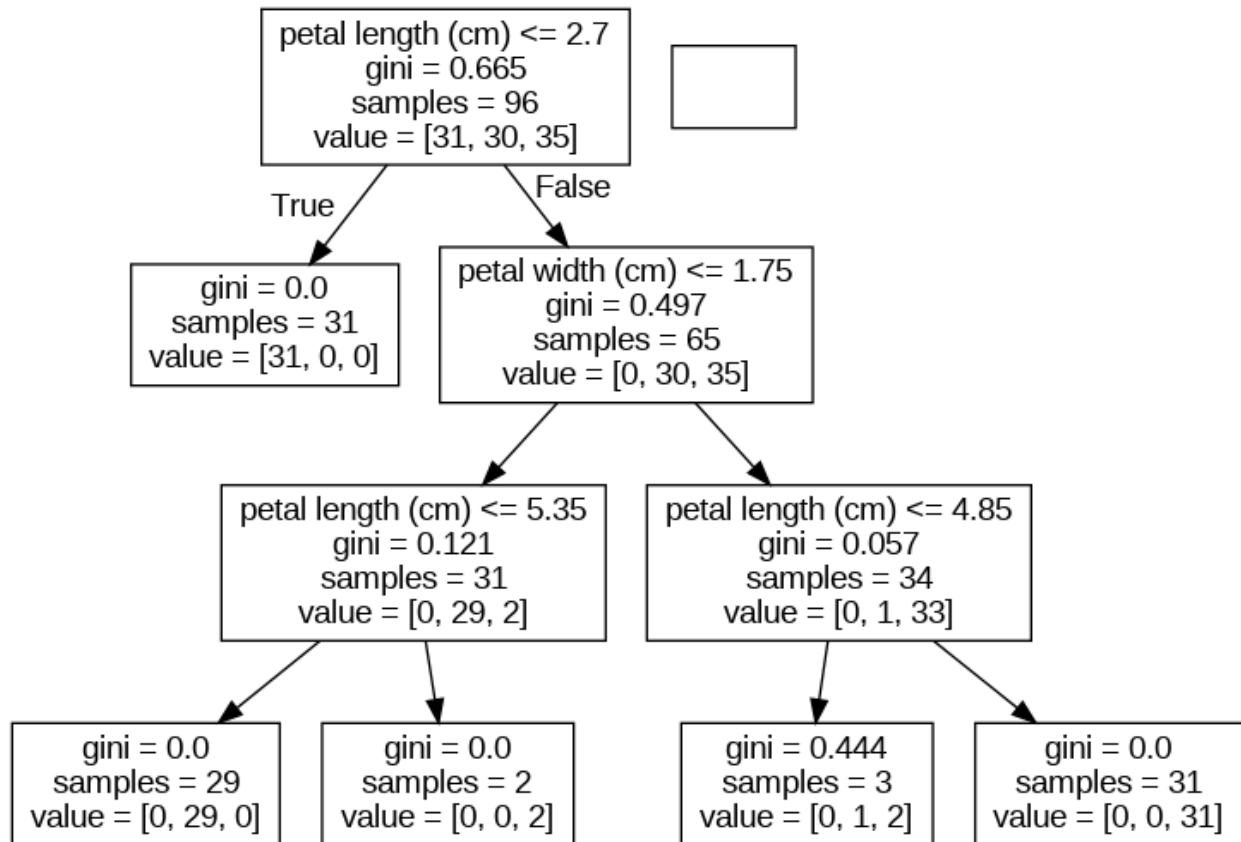
Requirement already satisfied: pydotplus in
/usr/local/lib/python3.11/dist-packages (2.0.2)
Requirement already satisfied: pyparsing>=2.0.1 in
/usr/local/lib/python3.11/dist-packages (from pydotplus) (3.2.1)

```

```

features = X.columns
dot_data = export_graphviz(dt, out_file=None, feature_names=features)
graph = pydotplus.graph_from_dot_data(dot_data)
Image(graph.create_png())

```



```

# show char the tree in graphics with plot_tree
from sklearn import tree

```

```

plt.figure(figsize=(20, 10))
tree.plot_tree(
    DecisionTreeClassifier(random_state=42, max_depth=3).fit(X_train,
t_train),
    feature_names=X.columns,
    class_names=t.unique().astype(np.str_),
    filled=True,
    rounded=True)

```

```

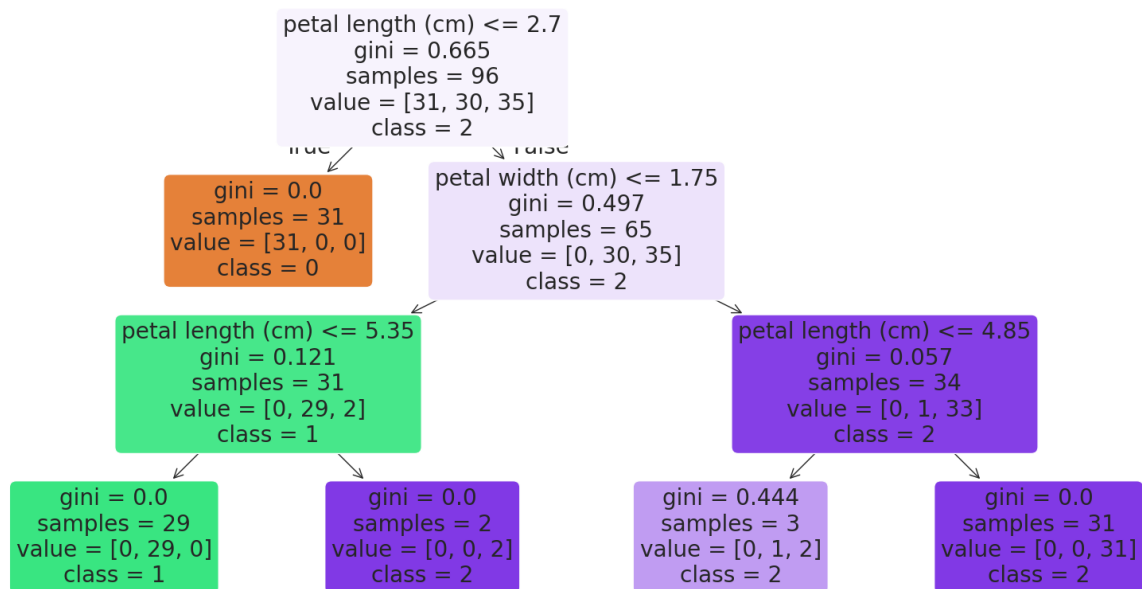
[Text(0.375, 0.875, 'petal length (cm) <= 2.7\ngini = 0.665\nsamples =
96\nvalue = [31, 30, 35]\nclass = 2'),
Text(0.25, 0.625, 'gini = 0.0\nsamples = 31\nvalue = [31, 0, 0]\
nclass = 0'),
Text(0.3125, 0.75, 'True '),
Text(0.5, 0.625, 'petal width (cm) <= 1.75\ngini = 0.497\nsamples =

```

```

65\nvalue = [0, 30, 35]\nclass = 2'),
Text(0.4375, 0.75, ' False'),
Text(0.25, 0.375, 'petal length (cm) <= 5.35\ngini = 0.121\nsamples =
31\nvalue = [0, 29, 2]\nclass = 1'),
Text(0.125, 0.125, 'gini = 0.0\nsamples = 29\nvalue = [0, 29, 0]\
nclass = 1'),
Text(0.375, 0.125, 'gini = 0.0\nsamples = 2\nvalue = [0, 0, 2]\nclass
= 2'),
Text(0.75, 0.375, 'petal length (cm) <= 4.85\ngini = 0.057\nsamples =
34\nvalue = [0, 1, 33]\nclass = 2'),
Text(0.625, 0.125, 'gini = 0.444\nsamples = 3\nvalue = [0, 1, 2]\
nclass = 2'),
Text(0.875, 0.125, 'gini = 0.0\nsamples = 31\nvalue = [0, 0, 31]\
nclass = 2')]

```



```

y_pred_train = dt.predict(X_train)
y_pred_test = dt.predict(X_test)
y_prob = dt.predict_proba(X_test)

print(f'Accuracy of decision tree (train):
{accuracy_score(y_pred_train, t_train)}')
print(f'Accuracy of decision tree (test): {accuracy_score(y_pred_test,
t_test)}')

Accuracy of decision tree (train): 0.9895833333333334
Accuracy of decision tree (test): 0.96875

print(f'Confusion Matrix (train):\n {confusion_matrix(t_train,
y_pred_train)}')

```



```
print(f'Confusion Matrix (test):\n {confusion_matrix(t_test,
y_pred_test)}')
```

```
Confusion Matrix (train):
```

```
[[31  0  0]
 [ 0 29  1]
 [ 0  0 35]]
```

```
Confusion Matrix (test):
```

```
[[15  0  0]
 [ 0  7  0]
 [ 0  1  9]]
```

```
print(classification_report(t_test, y_pred_test))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	15
1	0.88	1.00	0.93	7
2	1.00	0.90	0.95	10
accuracy			0.97	32
macro avg	0.96	0.97	0.96	32
weighted avg	0.97	0.97	0.97	32

```
dt = DecisionTreeClassifier(random_state=42)
```

```
params = {'max_depth': [2, 3, 4, 5],
          'min_samples_split': [2, 3, 4, 5],
          'min_samples_leaf': [1, 2, 3, 4,5]}
```

```
gsearch = GridSearchCV(dt, param_grid=params, cv=3)
```

```
gsearch.fit(X_train, t_train)
```

```
gsearch.best_params_
```

```
{'max_depth': 4, 'min_samples_leaf': 1, 'min_samples_split': 2}
```

```
# passing best params for hyperparameters tuning
```

```
dt = DecisionTreeClassifier(**gsearch.best_params_, random_state=42)
```

```
dt.fit(X_train, t_train)
```

```
y_pred_train = dt.predict(X_train)
```

```
y_prob_train = dt.predict_proba(X_train)[:,:1]
```

```
y_pred_test = dt.predict(X_test)
```

```
y_prob_test = dt.predict_proba(X_test)[:,:1]
```

```
print(f'Accuracy of decision tree (train):
```

```
{accuracy_score(y_pred_train, t_train)}')
```

```
print(f'Accuracy of decision tree (test): {accuracy_score(y_pred_test,
t_test)}')
```

```
Accuracy of decision tree (train): 1.0
Accuracy of decision tree (test): 0.96875
```

```
print(f'Confusion Matrix (train):\n {confusion_matrix(t_train,
y_pred_train)}')
print(f'Confusion Matrix (test):\n {confusion_matrix(t_test,
y_pred_test)}')
```

```
Confusion Matrix (train):
```

```
[[31  0  0]
 [ 0 30  0]
 [ 0  0 35]]
```

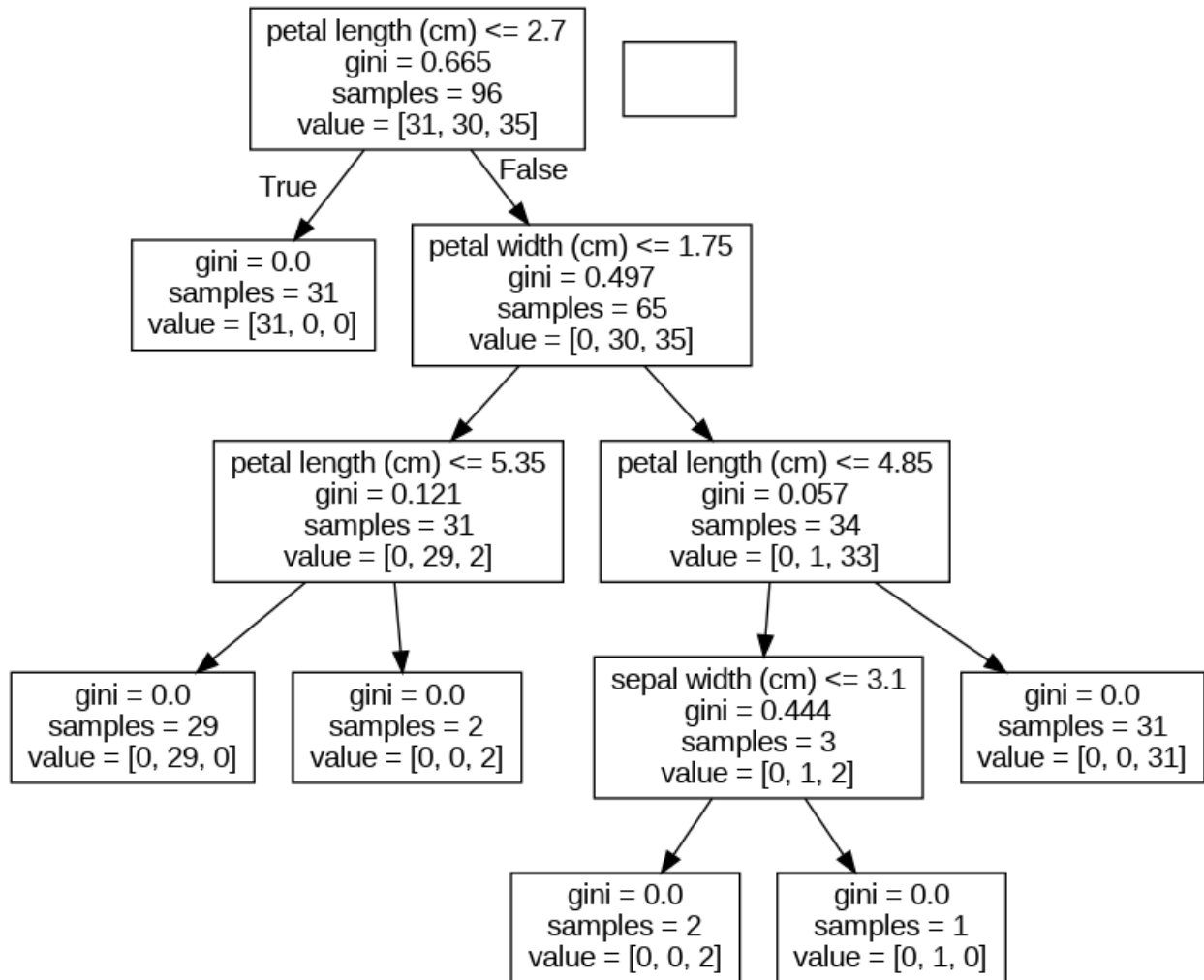
```
Confusion Matrix (test):
```

```
[[15  0  0]
 [ 0  7  0]
 [ 0  1  9]]
```

```
print(classification_report(t_test, y_pred_test))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	15
1	0.88	1.00	0.93	7
2	1.00	0.90	0.95	10
accuracy			0.97	32
macro avg	0.96	0.97	0.96	32
weighted avg	0.97	0.97	0.97	32

```
features = X.columns
dot_data = export_graphviz(dt, out_file=None, feature_names=features)
graph = pydotplus.graph_from_dot_data(dot_data)
Image(graph.create_png())
```



```
cross_val_score(DecisionTreeClassifier(**gsearch.best_params_,
random_state=42), X_train, t_train, cv=5).mean()
```

0.9473684210526315

Random Forest

The fundamental idea behind a random forest is to combine many decision trees into a single model.

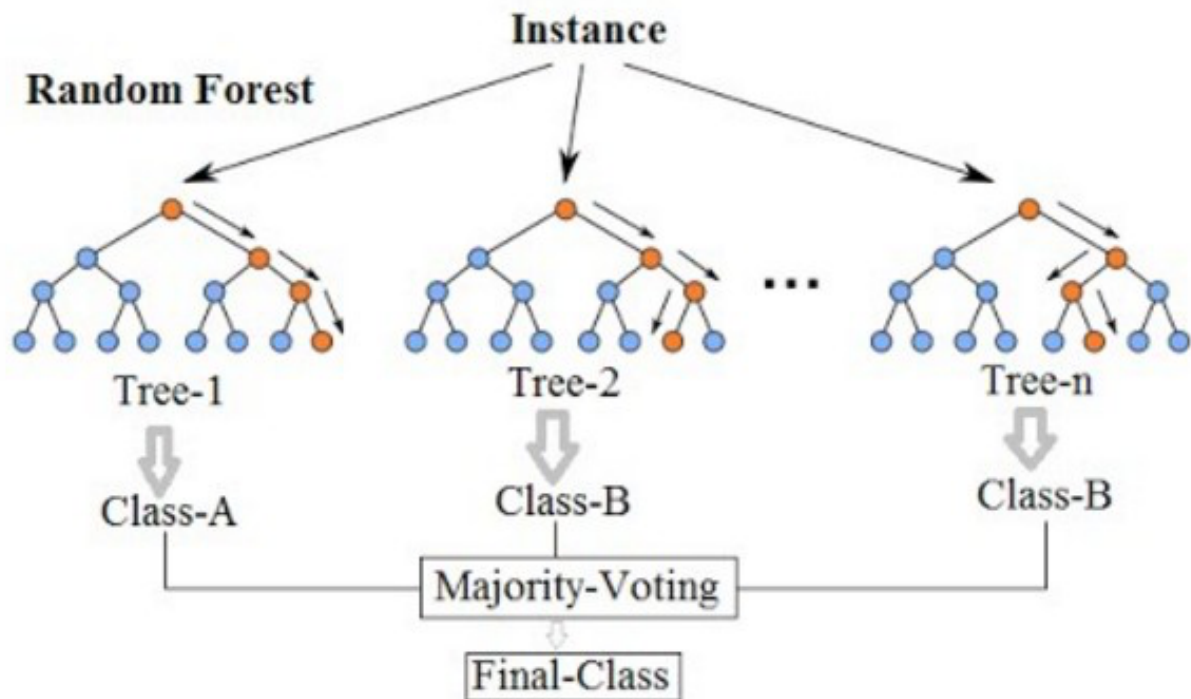
A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting.

Individually, predictions made by decision trees may not be accurate but combined together, the predictions will be closer to the mark on average.

Each decision tree in the forest considers a random subset of features and only has access to a random set of the training data points (Bootstrap).

Each decision tree in the forest may have a maximum depth.

Random Forest Simplified



Let's use Scikit-learn [RandomForestClassifier](#).

```
# show accuracy of random forest classifier on data
from sklearn.ensemble import RandomForestClassifier

print('RandomForestClassifier score:',
      cross_val_score(
          RandomForestClassifier(random_state=42, max_depth=3),
          X_train, t_train, cv=15).mean())
```

RandomForestClassifier score: 0.9571428571428572

More Information

More ways to encode categorical data:

[Smarter Ways to Encode Categorical Data for Machine Learning](#)

Theory explanation of Hashing Encoder:

[Don't be tricked by the Hashing Trick](#)

A practical explanation of Hashing Encoder:

[Introducing One of the Best Hacks in Machine Learning: the Hashing Trick](#)

Explanation of the differences between TF-IDF and N-gram:
[What is the difference between TF-IDF and n-gram?](#)

Example of Text Feature Extraction methods in Scikit-learn:
[Text feature extraction](#)

Guide for Decision Trees in Scikit-learn:
[Decision Trees](#)

Explanation of the differences between Gini and Entropy:
[Why are implementations of decision tree algorithms usually binary and what are the advantages of the different impurity metrics?](#)

Answer on what is better Gini vs. Entropy:
[When should I use Gini Impurity as opposed to Information Gain \(Entropy\)?](#)

Explanation on Random Forest:
[Random Forest Simple Explanation](#)

Example of Random Forest in Python:
[Random Forest in Python](#)